DOCUMENTACIÓN Y GUÍA TÉCNICA

Aplicación Xuxemons

Índice

Introducción **2**
Objetivos del proyecto **2**
Arquitectura de la aplicación **2**
Variables de entorno y su función **3**
¿Qué hacer cada vez que descarguemos Xuxemons del repositorio? **3**
Pasos para desplegar aplicación 4
Despliegue de la aplicación con Docker **4**
Descripción de los contenedores 5
1. Contenedor Frontend (Angular + Node.js) 5
2. Contenedor Backend (Laravel + PHP) 5
3. Contenedor Database (MySQL) 5
Red interna y comunicación 5
Ventajas del uso en Xuxemons 5
Requisitos para el despliegue 6
Requisitos de Hardware 6
Requisitos de Software 6
3. Configuración para el Despliegue (Docker Compose) 6
4. Puertos Libres 6
Contenido docker-compose.yml 7
Comandos 7
Imágenes de los resultados de cada comando 7
Estructura de carpetas del proyecto **8**
Tecnologías utilizadas y justificación **9**
Angular 10
Laravel 10
MySQL 10
Docker 11
Otras Tecnologías Complementarias 11
Accesibilidad **12**
Navegación Semántica y Estructural 12
Atributos ARIA 12
Accesibilidad Visual y Contraste 12
Formularios y Controles de Usuario 13
Autenticación y Autorización con JWT **13**
Gestión de errores y feedback **13**
Metodología de Trabajo **13**
Diagrama casos de uso **14**
Diagrama entidad-relación **16**

Introducción

Xuxemons es una aplicación web gamificada en la que los usuarios pueden coleccionar, gestionar y hacer crecer criaturas virtuales llamadas xuxemons. El sistema combina mecánicas de juego (como evolución, alimentación, inventario y colección) con funcionalidades sociales como amigos, intercambios, batallas y chat.

Los usuarios coleccionan xuxemons, los hacen crecer con xuxes, gestionar inventarios y pueden interactuar con otros jugadores mediante amigos, intercambios, batallas y chat, definiendo así un ecosistema completo de progresión e interacción.

La aplicación utiliza una arquitectura moderna basada en Laravel para el backend, Angular para el frontend y Docker para el despliegue, permitiendo un entorno modular, escalable y fácil de mantener. Además, incorpora funcionalidades como enfermedades, vacunas, recompensas diarias, seguridad con JWT y chat instantáneo, tal y como especifica el dossier.

Objetivos del proyecto

El proyecto Xuxemons tiene como finalidad crear una plataforma web gamificada en la que los usuarios se registren, coleccionen xuxemons y gestionen recursos dentro de un ecosistema interactivo. El objetivo de esta aplicación es que los jugadores coleccionen criaturas, gestionen su inventario con sus recursos e interactúen entre ellos, definiendo así una experiencia completa de juego e interacción social.

Los objetivos específicos del proyecto son:

- Permitir la autenticación de usuarios (registro, inicio de sesión y gestión del perfil).
- Gestionar xuxemons y recursos, incluyendo inventario, colección y evolución de xuxemons.
- Implementar mecánicas de juego avanzadas, como enfermedades, vacunas y recompensas diarias.
- Facilitar la interacción social entre jugadores a través de amigos, intercambios, batallas y chat.
- Garantizar un entorno seguro y escalable, utilizando JWT para la autenticación y Docker para el despliegue.
- Ofrecer una arquitectura moderna y modular, basada en Laravel (backend), Angular (frontend) y contenedores Docker.

Arquitectura de la aplicación

La aplicación sigue una arquitectura de tres capas separadas en contenedores Docker independientes que se comunican entre sí a través de una red interna:

El usuario accede desde el navegador al frontend Angular (puerto 4200). Cuando necesita datos, Angular lanza peticiones HTTP a la API REST de Laravel (puerto 8000), incluyendo el token JWT en cada cabecera. Laravel procesa la lógica, consulta MySQL (puerto 3306) si

es necesario, y devuelve la respuesta en formato JSON. Angular recibe la respuesta y actualiza la interfaz.

El flujo que sigue Xuxemons:

Navegador Angular (4200) Laravel API (8000) MySQL (3306)

Ninguna capa accede directamente a otra saltándose el intermediario. El frontend nunca toca la base de datos directamente, y la base de datos nunca es accesible desde fuera de la red Docker.

Variables de entorno y su función

Las variables de entorno permiten configurar la aplicación sin modificar el código fuente. En Xuxemons se definen principalmente en el archivo .env del backend (Laravel), que nunca se sube al repositorio por seguridad (Así que hay que poner estos valores cada vez que descarguemos el proyecto en un nuevo PC).

Variable	Valor	Función
DB_CONNECTION	mysql	Motor de base de datos utilizado
DB_HOST	db	Hostname del contenedor de base de datos en la red Docker
DB_PORT	3306	Puerto de conexión a MySQL
DB_DATABASE	xuxemons_db	Nombre de la base de datos
DB_USERNAME	xuxemons_user	Usuario de MySQL
DB_PASSWORD	1234	Contraseña de MySQL

¿Qué hacer cada vez que descarguemos Xuxemons del repositorio?

Hay que implementar las variables anteriores. Para eso vamos a seguir el siguiente procedimiento con los siguientes comandos:

1. Desde la carpeta /backend, hay que ejecutar los siguientes comandos:

Paso	Comando	Uso
1	cp .env.example .env	Copia el contenido de .env.example al archivo .env

2	composer install	Instala el gestor de dependencias de Laravel
3	php artisan key:generate	Genera una clave aleatoria y segura para la aplicación. Laravel la usa internamente para cifrar cosas como cookies, sesiones y datos sensibles. Sin ella, Laravel no funciona correctamente y lanza un error.

2. El .env debe tener las variables definidas que vimos en el apartado anterior.
3. Desplegar y ejecutar aplicación y migraciones, en el siguiente subapartado.

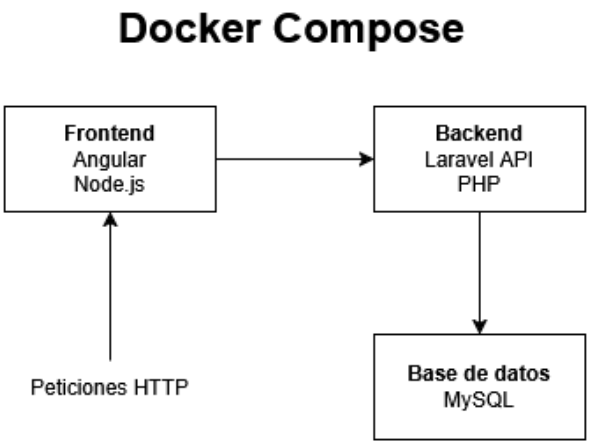
Pasos para desplegar aplicación

1. **Descomprimir** el proyecto y **acceder** a la carpeta **raíz** mediante la terminal.
2. Ejecutar el comando ***docker compose up -d***.
3. Una vez levantados los contenedores, ejecutar:
   docker compose exec backend php artisan migrate --seed.
4. Acceder a la aplicación a través de **http://localhost:4200**.

Aunque no se mencione explícitamente, estos pasos también se pueden hacer mediante Docker Desktop.

Despliegue de la aplicación con Docker

La aplicación Xuxemons se despliega mediante Docker Compose, que permite ejecutar el frontend, el backend y la base de datos en contenedores independientes pero conectados entre sí a través de una red interna. Esta arquitectura facilita la portabilidad, la reproducibilidad del entorno y el despliegue en cualquier máquina sin necesidad de instalar dependencias manualmente.



Descripción de los contenedores

1. Contenedor Frontend (Angular + Node.js)

- Compila y sirve la aplicación Angular.
- Expone el puerto (normalmente 4200).
- Se comunica con el backend mediante peticiones HTTP.
- Incluye Node.js para ejecutar Angular CLI.

2. Contenedor Backend (Laravel + PHP)

- Ejecuta la API REST de Laravel.
- Gestiona la lógica del juego, autenticación JWT, inventario, Xuxemons, batallas, chat, etc.
- Expone el puerto (normalmente 8000).
- Se conecta a la base de datos mediante la red interna de Docker.

3. Contenedor Database (MySQL)

- Almacena toda la información persistente:
    - Usuarios
    - Xuxemons
    - Inventarios
    - Amistades y solicitudes
    - Mensajes del chat
    - Parámetros del juego
- Utiliza volúmenes Docker para mantener los datos entre reinicios.

Red interna y comunicación

Docker Compose crea automáticamente una red bridge interna donde:

- El frontend puede comunicarse con backend usando el hostname backend.
- El backend puede comunicarse con database usando el hostname database.
- Los contenedores no exponen servicios hacia fuera salvo los puertos definidos explícitamente.

Esto garantiza un entorno seguro y aislado.

Ventajas del uso en Xuxemons

- Entorno de desarrollo y producción idéntico.
- Eliminación de problemas de dependencias locales.
- Escalabilidad y modularidad.
- Reinicio rápido de servicios.
- Persistencia de datos mediante volúmenes.
- Despliegue simplificado con una sola orden: *docker compose up -d*.

Requisitos para el despliegue

Requisitos de Hardware

Docker consume recursos ya que levanta contenedores independientes (en tu caso: Backend, Frontend y Base de Datos).

- **Procesador (CPU):** Procesador de 64 bits con soporte para virtualización. Debes activarlo en la BIOS/UEFI de tu placa base (no suele aparecer activado por defecto y por lo tanto puede dar error).
- **Memoria RAM:**
  - **Mínimo:** 4 GB de RAM.
  - **Recomendado:** +8 GB.
- **Almacenamiento:** Al menos 20 GB de almacenamiento.

Requisitos de Software

- **Windows:**
  - Windows 10 o 11.
  - Es obligatorio tener instalado el componente WSL 2 (Windows Subsystem for Linux) para un rendimiento óptimo.
- **Linux:**
  - Cualquier distribución moderna (Ubuntu 22.04+, Debian, Fedora…).
  - Es el SO más recomendable para utilizar docker.
- **macOS:**
  - macOS 12 o superior. Soporta procesadores de Intel y Apple Silicon (M1/M2/M3).

3. Configuración para el Despliegue (Docker Compose)

Para usar el archivo docker-compose.yml que tienes preparado:

- **Docker Engine:** El motor principal que ejecuta contenedores.
- **Docker Compose:** La herramienta para gestionar varios contenedores a la vez (suele venir incluido con Docker Desktop o como plugin docker-compose-plugin).
- **Conexión a Internet:** Necesaria la primera vez para descargar las imágenes oficiales de:
  - node: 20-alpine.
  - angular.
  - mysql: 8.0.
  - compose (para Laravel).
  - dependencias de Laravel y PHP.

4. Puertos Libres

Tu aplicación necesita que los siguientes puertos no estén ocupados por otros programas en el ordenador:

- **8000:** Para el Backend (Laravel).
- **4200:** Para el Frontend (Angular) (y para probar el funcionamiento real de la aplicación).
- **3306:** Para la Base de Datos (MySQL).

Contenido docker-compose.yml

El archivo utiliza la versión 3.9 de Docker Compose para orquestar un entorno de tres servicios interconectados en una red privada denominada app-network:

Servicio backend (Laravel):

- **Construcción:** Se genera a partir de un Dockerfile ubicado en la carpeta ./backend.
- **Puerto:** Expone el puerto 8000 para el acceso a la API.
- **Variables de Entorno:** Configura la conexión automática a la base de datos MySQL (xuxemons_db) usando las credenciales definidas.
- **Dependencia:** Está configurado para esperar a que el servicio de base de datos (db) esté listo antes de iniciar.

Servicio frontend (Angular):

- **Imagen:** Utiliza node:20-alpine para un entorno ligero.
- **Puerto:** Expone el puerto 4200 para el acceso a la interfaz web.
- **Comando:** Ejecuta npm start con soporte para Hot Module Replacement (HMR) y polling para detectar cambios de archivos en tiempo real durante el desarrollo.

Servicio BD (MySQL):

- **Imagen:** Versión oficial de mysql:8.0.
- **Persistencia:** Utiliza un volumen llamado mysql-data para asegurar que los datos de los xuxemons no se pierdan al reiniciar los contenedores.

Comandos

- **Iniciar el entorno:** docker compose up -d
- **Detener el entorno:** docker compose down
- **Ver logs en tiempo real:** docker compose logs -f
- **Observar los contenedores ejecutados:** docker-compose ps

Imágenes de los resultados de cada comando

Apagar servidores (backend, frontend, BD)

```
E:\GitHub\Xuxemons>docker compose down
time="2026-04-30T23:53:14+02:00" level=warning msg="E:\\GitHub\\Xuxemons\\docker-compose.yml: the attribute `
version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 4/4
 ✓ Container xuxemons_angular    Removed                                                    0.4s
 ✓ Container xuxemons_laravel    Removed                                                    1.3s
 ✓ Container xuxemons_db         Removed                                                    1.1s
 ✓ Network xuxemons_app-network  Removed                                                    0.3s
```

Ejecutar servidores

```
E:\GitHub\Xuxemons>docker compose up -d
time="2026-04-30T23:53:45+02:00" level=warning msg="E:\\GitHub\\Xuxemons\\docker-compose.yml: the attribute `
version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 4/4
 ✓ Network xuxemons_app-network  Created                                                    0.0s
 ✓ Container xuxemons_db         Started                                                    0.4s
 ✓ Container xuxemons_angular    Started                                                    0.5s
 ✓ Container xuxemons_laravel    Started                                                    0.5s
```

Observar registros en tiempo real

```
E:\GitHub\Xuxemons>docker compose logs -f
time="2026-04-30T23:54:50+02:00" level=warning msg="E:\\GitHub\\Xuxemons\\docker-compose.yml: the attribute `
version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
xuxemons_angular  |
xuxemons_angular  | > frontend@0.0.0 start
xuxemons_angular  | > ng serve --host 0.0.0.0 --poll=500 --hmr
xuxemons_angular  |
xuxemons_angular  |
xuxemons_db       | 2026-04-30 21:53:46+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.45-
1.el9 started.
xuxemons_angular  | Warning: This is a simple server for use in testing or debugging Angular applications
```

```
xuxemons_db       | 2026-04-30T21:53:47.566350Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for c
onnections. Version: '8.0.45'  socket: '/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community Server - GP
L.
xuxemons_laravel  | 127.0.0.1 - - [30/Apr/2026:21:54:34 +0000] "OPTIONS * HTTP/1.0" 200 126 "-" "Apache/2.4.6
6 (Debian) PHP/8.3.30 (internal dummy connection)"
```

Observar contenedores

Ya que salen los tres contenedores (xuxemons_angular, xuxemons_db y xuxemons_laravel) significa que los contenedores necesarios para el funcionamiento de la aplicación se han iniciado correctamente.

```
E:\GitHub\Xuxemons>docker compose ps
time="2026-04-30T23:57:41+02:00" level=warning msg="E:\\GitHub\\Xuxemons\\docker-compose.yml: the attribute `version` is obsolet
e, it will be ignored, please remove it to avoid potential confusion"
NAME               IMAGE              COMMAND                  SERVICE    CREATED         STATUS         PORTS
xuxemons_angular   node:20-alpine     "docker-entrypoint.s…"   frontend   3 minutes ago   Up 3 minutes   0.0.0.0:4200->4200/tcp,
 [::]:4200->4200/tcp
xuxemons_db        mysql:8.0          "docker-entrypoint.s…"   db         3 minutes ago   Up 3 minutes   0.0.0.0:3306->3306/tcp,
 [::]:3306->3306/tcp
xuxemons_laravel   xuxemons-backend   "docker-php-entrypoi…"   backend    3 minutes ago   Up 3 minutes   0.0.0.0:8000->80/tcp, [
::]:8000->80/tcp
```

Estructura de carpetas del proyecto

El proyecto Xuxemons se organiza en una estructura modular que separa claramente el frontend, el backend y la configuración de despliegue con Docker. Esta organización facilita

el mantenimiento, la colaboración en equipo y el despliegue en entornos de desarrollo y producción.

A continuación se muestran las carpetas principales que se utilizan para el proyecto, las que contienen archivos que se han creado y las mismas carpetas que se han tenido que crear, que dentro contiene archivos también generados y desarrollados.

<table>
<tr><td rowspan="14">Xuxemons/</td><td colspan="3" rowspan="12">Frontend/src/app</td><td>admin</td></tr>
<tr><td>dashboard</td></tr>
<tr><td>interfaces</td></tr>
<tr><td>guards</td></tr>
<tr><td>no-autorizado</td></tr>
<tr><td>amigos</td></tr>
<tr><td>login</td></tr>
<tr><td>mochila</td></tr>
<tr><td>perfil</td></tr>
<tr><td>registro</td></tr>
<tr><td>Services</td></tr>
<tr><td>xuxemons</td></tr>
<tr><td rowspan="5">Backend/</td><td rowspan="2">app/</td><td>Http/Controllers</td><td>Auth</td></tr>
<tr><td colspan="2">Models</td></tr>
<tr><td rowspan="2">database/</td><td colspan="2">migrations</td></tr>
<tr><td colspan="2">seeders</td></tr>
<tr><td colspan="3">routes/</td></tr>
<tr><td colspan="3">config/</td></tr>
</table>

Tecnologías utilizadas y justificación

Xuxemons se ha desarrollado utilizando un conjunto de tecnologías modernas que permiten garantizar rendimiento, escalabilidad, mantenibilidad y facilidad de despliegue. Cada tecnología ha sido seleccionada en función de adecuación a los requisitos del proyecto descritos exigidos.

Angular

- Facilita la creación de interfaces complejas mediante componentes reutilizables.
- Ofrece formularios reactivos, ideales para validaciones de login, registro y edición de perfil.
- Permite gestionar estados y servicios para módulos como inventario, xuxedex, amigos o chat.
- Su sistema de routing es perfecto para una SPA con múltiples pantallas.
- Se integra fácilmente con interceptores para enviar el token JWT en cada petición.
- Proporciona un ecosistema sólido y mantenido por Google.

¿Por qué Angular?:
La aplicación requiere múltiples vistas, interacción constante con la API y actualizaciones dinámicas del estado del usuario, lo que encaja perfectamente con Angular.

Laravel

Descripción:

- Incluye un ORM (Eloquent) que simplifica el acceso y la gestión de la base de datos.
- Permite implementar autenticación con JWT de forma segura.
- Facilita la creación de controladores y rutas para módulos como usuarios, xuxemons, inventario, batallas, chat y amigos.
- Ofrece middleware para roles, permisos y validación de peticiones.
- Su arquitectura MVC favorece la mantenibilidad y escalabilidad del proyecto.
- Dispone de migraciones y seeders para gestionar la estructura de la base de datos.

¿Por qué Laravel?:

El backend debe gestionar lógica compleja (evolución, enfermedades, recompensas diarias, batallas, chat), y Laravel proporciona herramientas potentes para implementarla de forma clara y segura.

MySQL

Descripción:

- Permite definir relaciones entre usuarios, xuxemons, inventarios, amistades y mensajes.
- Ofrece integridad referencial mediante claves foráneas.
- Es estable, rápido y fácil de administrar.
- Se integra perfectamente con Eloquent ORM.
- Funciona sin problemas en entornos Docker.

¿Por qué MySQL?:

El proyecto requiere almacenar datos estructurados y relacionados (usuarios, criaturas, objetos, batallas, chat), lo que encaja perfectamente con un modelo relacional.

Docker

Descripción:

- Permite separar frontend, backend y base de datos en contenedores independientes.
- Facilita el despliegue con un solo comando (O desde un solo botón en su interfaz de escritorio).
- Evita problemas de dependencias locales.
- Asegura entornos reproducibles y consistentes.
- Permite persistencia de datos mediante volúmenes.

¿Por qué Docker?:

El proyecto requiere un entorno estable, fácil de desplegar y replicable por cualquier miembro del equipo, lo que Docker resuelve de forma óptima.

Otras Tecnologías Complementarias

Tecnología	Uso	Motivo
Node.js	Compilación y ejecución de Angular	Sin esto no se puede arrancar el servidor de desarrollo de Angular ni construir la aplicación para su uso.
Composer	Gestión de dependencias de Laravel	Xuxemons depende de paquetes externos como el ORM Eloquent, JWT y el sistema de migraciones.
JWT	Autenticación sin estado	Sin JWT hay que gestionar sesiones en el servidor, lo que complica el escalado y no encaja bien con una arquitectura de API REST separada del frontend.
Bootstrap	Estilos y componentes del frontend	Permite construir diseños CSS sin tener que escribir la lógica completa desde 0.
CSS Grid / Flexbox	Diseño de inventario, Xuxedex y layouts	Diseñar layouts de múltiples casillas de forma más sencilla.

Accesibilidad

Navegación Semántica y Estructural

Se ha utilizado HTML5 semántico para permitir que los lectores de pantalla interpreten correctamente la jerarquía de la página:

- **Roles y etiquetas:** Uso de etiquetas <header>, <main> y <footer>, además de roles específicos como role="main" y role="navigation".
- **Identificadores de contenido:** Implementación de id="main-content" para permitir el salto directo al contenido principal.
- **Jerarquía de encabezados:** Uso estructurado de <h1> y <h2> para facilitar la comprensión del esquema de cada sección.

Atributos ARIA

El código fuente incluye atributos avanzados para mejorar la experiencia de usuarios con asistentes de voz:

- **Aria-label:** Aplicado en botones críticos (como la navegación inferior) y en los "slots" de la mochila para describir contenido que es puramente visual (ej. *[attr.aria-label]="'Slot ' + ($index + 1)"*).
- **Aria-live (Regiones Dinámicas):** Se ha implementado aria-live="polite" en los contadores de la Xuxedex y mensajes de confirmación. Esto asegura que el usuario sea notificado automáticamente cuando el número de Xuxemons o el estado de una acción cambie, sin interrumpir su navegación.
- **Aria-hidden:** Se utiliza en iconos decorativos y emojis (aria-hidden="true") para evitar que el lector de pantalla lea caracteres innecesarios que no aportan valor informativo.

Accesibilidad Visual y Contraste

En los archivos CSS se han implementado medidas para usuarios con visión dificultada:

- **Clase visually-hidden:** Se utiliza para incluir texto que solo es detectable por lectores de pantalla, proporcionando contexto adicional en elementos que visualmente son solo iconos.
- **Doble codificación:** Los estados críticos (como el de "Mochila Llena") no solo se indican mediante el cambio de color a rojo, sino que se refuerzan con cambios en el grosor de la fuente (font-weight: 900) y etiquetas de texto descriptivo.
- **Indicadores de Foco:** Se han respetado y estilizado los estados de foco (:focus) para que los usuarios que navegan mediante teclado sepan siempre en qué elemento se encuentran.

Formularios y Controles de Usuario

- **Etiquetado de Inputs:** Todos los campos de entrada en el panel de Administración y los buscadores utilizan aria-label o etiquetas <label> vinculadas, garantizando que el propósito de cada campo sea claro.
- **Validación de errores:** Los mensajes de error y confirmación están integrados en el flujo semántico, permitiendo que el usuario reciba feedback inmediato sobre sus acciones (ej. "No tienes xuxes en la mochila").

Autenticación y Autorización con JWT

La aplicación utiliza JSON Web Tokens (JWT) para la seguridad. Cuando el usuario se loguea, el backend (Laravel) genera un token único. Angular almacena este token y lo envía en el Header de cada petición. Si el token no es válido o ha expirado, el middleware de Laravel bloquea el acceso a los datos de los Xuxemons.
Un usuario normal no puede acceder a las rutas de /admin del backend aunque tenga un token, porque el sistema verifica su nivel de permisos.

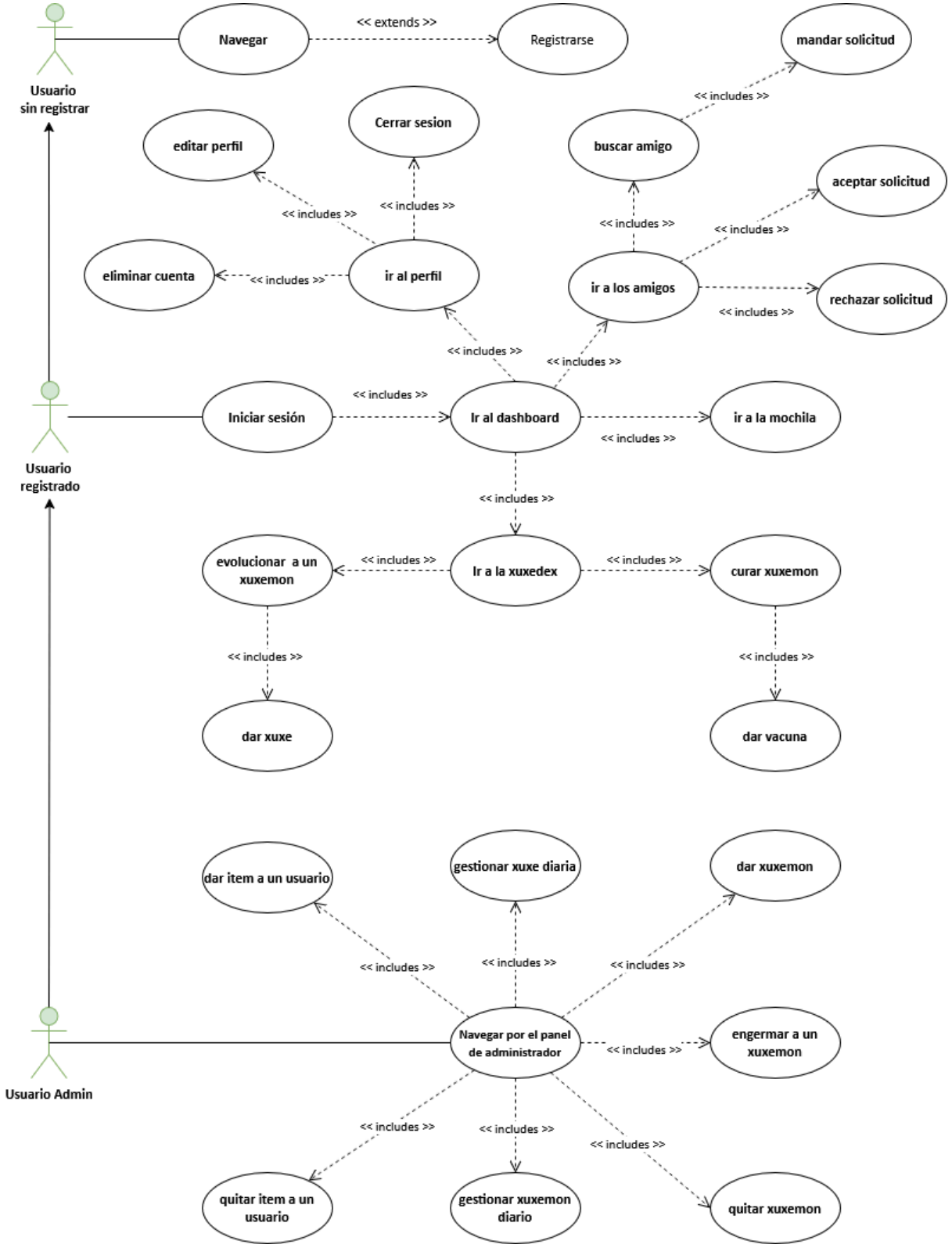
Gestión de errores y feedback

Se ha implementado un sistema de feedback proactivo. Si un usuario intenta alimentar a un Xuxemon sin tener dulces, el sistema no solo bloquea la acción, sino que muestra un mensaje de advertencia visual mediante un *role="status"* (detectable por lectores de pantalla). Lo mismo ocurre en el login si las credenciales son incorrectas.

Metodología de Trabajo

Hemos utilizado GitHub Desktop como sistema de control de versiones, trabajando con ramas para no interferir en el código principal mientras desarrollábamos nuevas funciones (como la mochila o el sistema de evolución). Por lo tanto, el proyecto es descargable desde su respectivo repositorio, disponible en la web de GitHub.

Diagrama casos de uso



El diagrama muestra tres tipos de perfiles y las acciones que puede realizar cada uno.

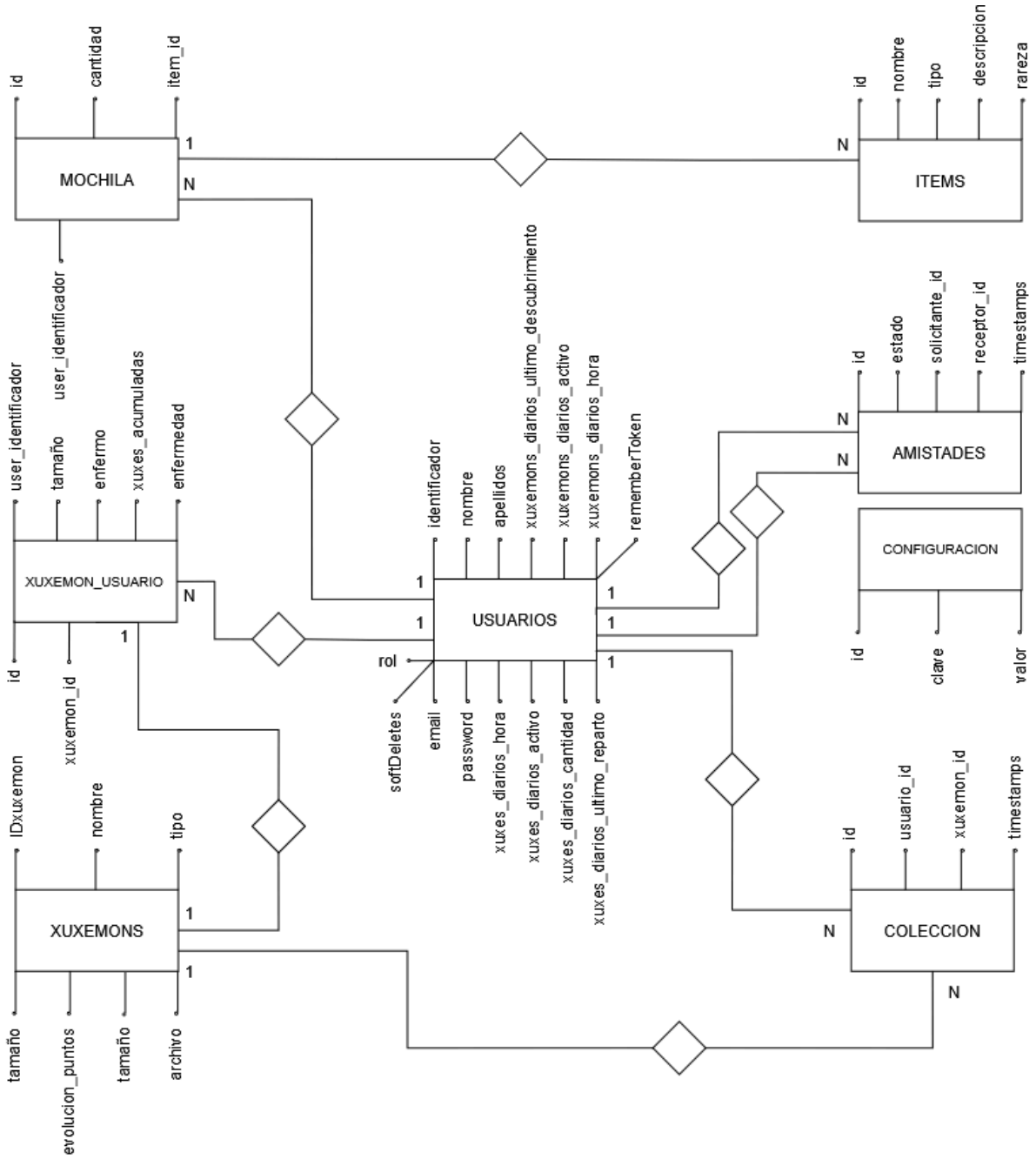
- **Usuario sin registrar** Solo puede navegar por la aplicación, y desde ahí tiene la opción de registrarse para acceder al resto de funcionalidades.

- **Usuario registrado** Es el actor principal del sistema. Una vez inicia sesión accede al dashboard, desde donde puede navegar a tres áreas principales: el perfil, los amigos y el contenido del juego.

  Desde este perfil puede editar su información personal, cerrar sesión o eliminar su cuenta. Desde la sección de amigos puede buscar otros jugadores, mandar solicitudes de amistad, y aceptar o rechazar las que recibe. Desde el dashboard accede también a la mochila (inventario de ítems) y a la Xuxedex (colección de Xuxemons), donde puede evolucionar a sus Xuxemons dándoles xuxes, o curarlos usando vacunas de su mochila.

- **Usuario Admin** Accede a un panel de administración desde el que puede gestionar el juego de forma global: dar ítems a usuarios o quitárselos, dar o quitar Xuxemons, enfermar a un Xuxemon, y configurar tanto la xuxe diaria (cantidad y hora de entrega) como el Xuxemon diario (hora de descubrimiento).

Diagrama entidad-relación



La estructura de la base de datos que tiene la aplicación.

Entidades principales:

- **USUARIOS:** Es la entidad central del sistema. Almacena:
  - Datos personales
  - Estado del juego
  - Datos de Xuxemons
  - Sistema

- **XUXEMONS:** Las criaturas coleccionables del juego. En esta tabla se guarda cada entidad de forma única.
- **XUXEMON_USUARIO: I**nstancia concreta de un Xuxemon perteneciente a un usuario. Dicho xuxemon es uno que esté ya definido en la tabla Xuxemons.
- **MOCHILA:** Inventario del jugador. Se guardan los objetos que el jugador obtenga a partir de los que existan en la tabla Items.
- **ITEMS:** Objetos del juego. Identifica de forma única cada objeto que puede haber en el juego y que puede obtener el jugador.
- **COLECCION:** Registro de Xuxemons coleccionados por el usuario.Similar a la funcionalidad de Tabla_Usuario, pero aparte pudiendo ver cuando el usuario coleccionó ese xuxemon.
- **AMISTADES:** Permite que haya una red de amigos en la aplicación. Esto creando una relación entre dos o más usuarios. Sería útil sobre todo a la hora de implementar las batallas y chat en la mecánica del juego, cosa que está en desarrollo, pero no es funcional.
- **CONFIGURACION:** Guarda los parámetros configurables del juego como pares clave-valor. En lugar de tener esos valores fijos en el código, se guardan en la base de datos para que el administrador pueda cambiarlos sin tocar el código fuente. Por ejemplo, si el admin quiere que se necesiten 5 xuxes en lugar de 3 para evolucionar de Pequeño a Mediano, solo cambia el valor en esa tabla y el juego lo aplica automáticamente.